STUDENT DATA ANALYSIS USING SQL

SQL QUERIES OVERVIEW

- Performed data analysis using SQL queries

Used:

- Aggregations
- Joins
- Subqueries
- Window functions

TOTAL NUMBER OF STUDENTS IN DATABASE

```
SELECT COUNT(*) AS Total_Student
FROM Students;
```

Total_Student
10

OVERALL ACADEMIC PERFORMANCE

- SELECT ROUND(AVG(marks),2) AS avg_marks
  FROM Marks;

	avg_marks
▶	82.10

STUDENT-WISE AVERAGE MARKS

```
SELECT student_id,
ROUND(AVG(marks),2) AS
avg_marks
FROM Marks
GROUP BY student_id;
```

student_id	avg_marks
3	75.50
4	83.50
5	73.50
6	77.50
7	89.00
8	92.00

LOW ATTENDANCE STUDENTS

```
SELECT s.name, a.attendance_percentage
FROM Students s
JOIN Attendance a ON s.student_id = a.student_id
WHERE a.attendance_percentage < 75;
```

	name	attendance_percentage
▶	amit	70.25
	neha	60.00

STUDENTS PER COURSE

```
SELECT c.course_name, COUNT(s.student_id) AS total_students
FROM Students s
JOIN Courses c ON s.course_id = c.course_id
GROUP BY c.course_name;
```

course_name	total_students
Data Science	3
Web Development	2
Machine Learning	1
Cloud Computing	2
Cybersecurity	2

SUBJECT-WISE AVERAGE MARKS

```
SELECT subject, ROUND(AVG(marks),2) AS avg_marks
FROM Marks
GROUP BY subject;
```

subject	avg_marks
SQL	77.50
Python	87.50
HTML	83.00
CSS	82.00
Machine Learning	79.00
Statistics	88.50

HIGHEST MARKS PER SUBJECT

- SELECT subject, max(marks) AS highest_marks
  FROM Marks
  GROUP BY subject;

subject	highest_marks
SQL	85
Python	90
HTML	88
CSS	82
Machine Learning	95
Statistics	89

HIGH-PERFORMING GROUP

```
SELECT student_id, ROUND(AVG(marks),2) AS avg_marks
FROM Marks
GROUP BY student_id
HAVING AVG(marks) > (SELECT AVG(marks) FROM Marks);
```

student_id	avg_marks
1	87.50
4	83.50
7	89.00
8	92.00

RANK-BASED PERFORMANCE

```
SELECT student_id, avg_marks,
RANK() OVER (ORDER BY avg_marks DESC) AS student_rank
FROM (
    SELECT student_id,
 AVG(marks) AS avg_marks
    FROM Marks
    GROUP BY student_id
) t;
```

student_id	avg_marks	student_rank
8	92.0000	1
7	89.0000	2
1	87.5000	3
4	83.5000	4
9	82.0000	5
10	80.5000	6

COMPLETE ACADEMIC VIEW

```
SELECT s.name, c.course_name, m.subject, m.marks
FROM Students s
JOIN Courses c ON s.course_id = c.course_id
JOIN Marks m ON s.student_id = m.student_id;
```

name	course_name	subject	marks
mona	Data Science	SQL	85
mona	Data Science	Python	90
rahul	Web Development	HTML	78
rahul	Web Development	CSS	82
anita	Data Science	Machine Learning	63
anita	Data Science	Statistics	88

TOP STUDENT PER SUBJECT

```
SELECT *
FROM (
    SELECT subject, student_id, marks,
    RANK() OVER (PARTITION BY subject ORDER BY marks DESC) AS
rnk
    FROM Marks
) t
WHERE rnk = 1;
```

subject	student_id	marks	rnk
Cloud Fundamentals	4	92	1
Cryptography	5	67	1
CSS	2	82	1
Cybersecurity	5	80	1
Ethical Hacking	10	83	1
HTML	7	88	1

TOP STUDENT PER SUBJECT

```
SELECT student_id, subject, marks,
SUM(marks) OVER (PARTITION BY student_id ORDER BY mark_id)
AS running_total
FROM Marks;
```

student_id	subject	marks	running_total
1	SQL	85	85
1	Python	90	175
2	HTML	78	78
2	CSS	82	160
3	Machine Learning	63	63
3	Statistics	88	151